

The judges scoring, note	Team number: 20007	The judges scoring, note
The judges scoring, note	Problem: A	The judges scoring, note

Title: The 2048 game analysis and promotion

Abstract

In this paper, the 2048 games, expressed the strategy and the establishment of customs clearance of about 2048 game specific idea and algorithm of the model. By constructing universal model mobile number need to complete 2048 points have been analyzed and the probability of success, and through the SPSS and Java and so on software to verify the effectiveness of the two indicators. Using the model information first from $1 * 1$ to $4 * 4$ squares can achieve maximum value is analyzed, according to analysis of the data, draw the corresponding recursive relations, from the general to the specific integration out of the $N * N$ grid can achieve the maximum score: $2^{(N * N + 1)}$.

In view of the problem a, setting up two models, Respectively by AI algorithm (With a 2048 source code, to analyze its component framework, in order to further upgrading Game 2048 a flow of the game, and completing the PC platform extension) model and the Monte Carlo method to integrate the model to calculate the needed to reach 2048 points probability of success. Model one, by double game theory model, the Mini max algorithm by using the AI algorithm and Alpha Beta pruning enlightened four indicators, to inspire indicators in the Java program is running success probability and mobile number. According to the survey of 90 players to get some statistical data, then use SPSS software for processing and analysis, practical results to verify the validity of the theoretical results. After using Monte Carlo method on the basis of the successful probability analysis need to finish the game out of the target mobile number by using SPSS program for two methods to make the model of mobile number is obtained by practical data processing, the calculated results to verify the model is feasible and effective.

In view of the problem, this article through the analysis of the problem a result and find 2048 play, and then from simple to complex, from complex to general (from $1 * 1$ to $4 * 4$, $4 * 4$ to $N * N$) reach the highest scores, thus find the recursive equation, and obtain the general expression. In the form of $4 * 4$, continuing to play, then can play to the largest number of $2^{17}=131072$. Due to play in the end is the probability that occurs when a number is uncertainty, is 2 or 4, so the results of the maximum score is uncertain. Appearing in the last 2 cases of maximum score is $2^{(N * N)}$, the final Numbers for 4 cases of maximum score is $2^{(N * N + 1)}$, and through the actual operation get results and calculation results consistent with each other.

Key words: Mobile number, Probability of success, AI algorithms, Monte Carlo method, SPSS

Table of Contents

1. The problem repeat.....	- 2 -
2. The model basic assumptions	- 2 -
3. The instructions of symbols.....	- 2 -
4. The problem analysis.....	- 3 -
4.1 The analysis of the problem	- 3 -
4.2 The analysis of the problem two	- 4 -
5 Building and solving of the model	- 4 -
5.1 Problem a model and solving	- 4 -
5.1.1 AI Algorithms.....	- 4 -
5.1.2 The 2048 game analysis and promotion	- 8 -
5.1.3 Monte Carlo method and 2048 games	- 11 -
5.2 The question of Establishment and solving the problem.....	- 14 -
5.2.2 Maximum Solutions Analyze - Finally number 4	- 15 -
6. Model's Evaluation and Popularization.....	- 17 -
6.1 The First Model's Evaluation and Popularization	- 17 -
6.2 The Second Model's Evaluation and Popularization	- 17 -
7. Reference	- 18 -
8. Appendix	- 18 -

The 2048 game analysis and promotion

1. The problem repeat

Game 2048 is a new and popular intellectual game, the game process need to control all the squares to the same direction, having the same number of squares and merge into them, and each time slip to complete the blank grid will appear random 2 or 4, over and over again slip operations, end up with a 2048 - square lattice is the victory, if full final with grid, no 2048 the game would be lost.

The first question: it is because the rules of the game are simple that very popular with people, but no one can give a general model to make it very easy to do 2048, please give a general model to 2048, and using the mobile number and probability of success two indicators to verify the validity of the model.

The second question: as long as the success of 2048 players all know, when reach 2048, the game still can continue, but how much the biggest can achieve? This needs to be discussed with the learned knowledge to prove, if the grid number from $4 * 4$ expand to $N * N$, how much can achieve the maximums values?

2. The model basic assumptions

Hypothesis that a problem in the first model to complete Game 2048 has the same number is preferred in the process of merger, and the direction of the finger sliding digital into consideration.

Hypothesis two in a second model using Monte Carlo method will the length l assumptions for the present digital location, and it will point out the location of the digital distance sliding for a hypothesis for the length of the square a .

Hypothesis three in the process of problem 2 maximum run by adopting the method of "snake glyph" move, so just can achieve the maximum several mobile layout according to the square.

3. The instructions of symbols

N	single sample case number
n	The number of mobile digital reach their goals
Sig	The probability of bilateral T test
df	Freedom

a	Sliding into position
m	The number of sliding
x	The random number
σ_i	Mobile number of variance
μ	Mobile number of expectations
E	The combined total of expectations the parameter
γ_i	Absolute tolerance
y	Total combined parameter
C	Tolerance level
P	Probability of success
Q	Deviating from the target value of the average losses
L	The number of losses

4. The problem analysis

4.1 The analysis of the problem

Deal with the problem of the first step is the nature of the Game 2048 and the rules of the game have good understanding. Problem a set up two models, in view of the model a, using AI algorithms *Mini max* algorithm and *Alpha – Beta* pruning two content are discussed. *Mini max* algorithm is a kind of pessimism algorithm, first consider when making a decision is to select each strategy is the worst possible result, and then choose the best results from the worst results, then make the corresponding pattern number. *Alpha – Beta* pruning analysis obtained four indicators, AI algorithms through the *Java* running can work out the probability of successful synthesis of 2048 more than 90% (91.739%), but the probability is not high for synthesis of 4096 or even 8192. Model 2, on the premise of a certain, using

Monte Carlo method, mobile digital decision depends on the level of tolerance, to remember $a(a > 0)$, so the steps need to complete the game as follows: $p = \frac{2l}{a\pi}$. In

this regard, conducted a survey of 100 game players, getting the 92 game players feedback effective numerical and 83 players use *SPSS* to get customs clearance of numerical solution, finally get the mean and the number of mobile probability model. And then combined with information in the attachment, better to demonstrate the effectiveness of the model.

4.2 The analysis of the problem two

After you get 2048, the game still can continue to play, in order to get the maximum value, respectively from the Game 2048 involved in score of source and the rules of the game in the creation of a certain rule. Through the analysis of the source code, the analysis to the maximum solution from probability into consideration, consider in two different conditions. The first kind of circumstance is the last one before the full grid number is 2, the second is before the full grid happens to be a final number 4, respectively $2^{(N * N)}$ and $2^{(N * N + 1)}$ 。

5 Building and solving of the model

5.1 Problem a model and solving

5.1.1 AI Algorithms

Query to understand, *AI* algorithms with the game, 2048 can be even larger probability higher than 90% probability to win the Game 2048. 2048 the *AI* algorithms will be mainly divided into two parts, the first part introduces the use of basic algorithms, namely the *Mini max* and *Alpha – Beta* pruning; The second part analyses how we the operation of the specific algorithm implementation *AI* - 2048.

Model algorithm analysis, basic algorithm:

Game 2048 in essence can abstract as "the information is consistent" double game model, the players use fingers can move any four directions, after the game screen in the space of any of the numbers 2 or 4. "the information is consistent" refers to the right as at any time the two sides to the pattern of the information content is completely consistent, game strategies relying only on the next to the pattern of reasoning.

Mini max Algorithm

Solving the problem of the game theory method is numerous, in terms of natural idea generally is the ideal pattern into a tree, the tree has many branch and node, every node represents a kind of pattern tree, *Mini max* is also interlinked, it

through the pattern before father pattern to determine the next step for the pattern of root tree search pattern choice. The most basic core of all pattern tree search algorithm is assessment of the value of each pattern.

Basic steps are as follows:

(1) First of all, determining the maximum search depth H , and to know that can reach the final pattern, also can be in a pattern

(2) In the maximum depth for H of the pattern tree leaf node, using the predefined value evaluation function to evaluate the value of leaf nodes.

(3) From below to above as a leaf node assignment. Among $\max$ the child nodes, including node minimum take child node

(4) Every time we turn (at this time will be in the pattern of a $\max$ of the tree node), select nodes is equal to the $\max$ value of the child node path.

Alpha – Beta Pruning:

Starting from the root node, using the *Alpha – Beta* pruning method of steps:

(1) The first of the root nodes *Alpha* and *Beta* initialized respectively $-\infty$, and $+\infty$

(2) The first search the second layer first child (not a leaf node), and so *Alpha* and *Beta* inherited from his father node, so they also respectively $-\infty$, and $+\infty$.

(3) Searching the third layer first child, *Alpha* and *Beta* the same as above, respectively $-\infty$, and $+\infty$.

(4) Searching the fourth floor to the leaf node, and adopt the evaluation function to get the node value is 1, as shown in figure 1

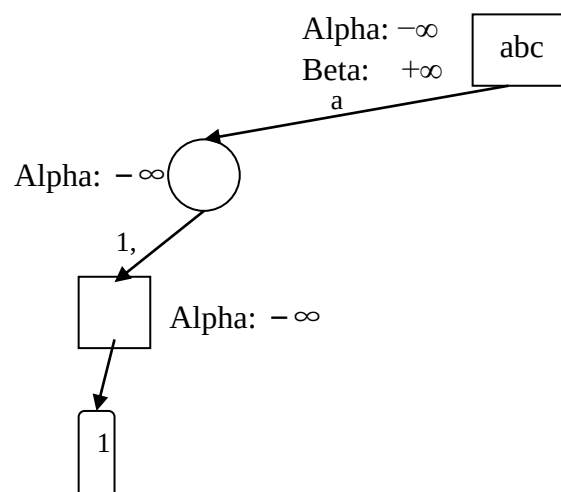


Figure 1 *Alpha – Beta* pruning one

(5) The leaf nodes of the parent node as $\max$ the node, thus updating *Alpha* its

value is 1, said the node values, the minimum value of 1.

(6) To see another child node, its value is 4, greater than the current value, so will *Alpha* update of 4

(7) Then the third layer finish the entire leftmost node finish search, as max nodes, updating its true value for the current *Alpha* value of 4.

(8) Due to its parent (the second leftmost node) for the *min* node, thus to update its parent *Beta* value of 4, said this node for a maximum of four values, as shown in figure 2.

(9) To search for the second layer of the leftmost node have a second child and its subtree, according to the above logic, getting a value of 5 (note the second layer of *Beta* the leftmost node values to pass to the child). Since 5 is bigger then 4, do not update node values, as shown in figure 3.

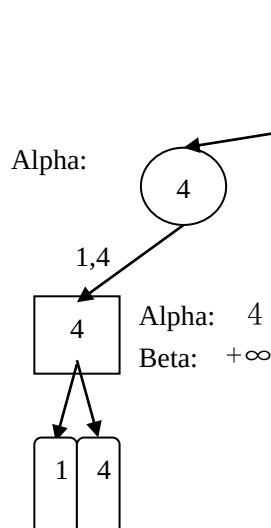


Figure 2 *Alpha – Beta* pruning two

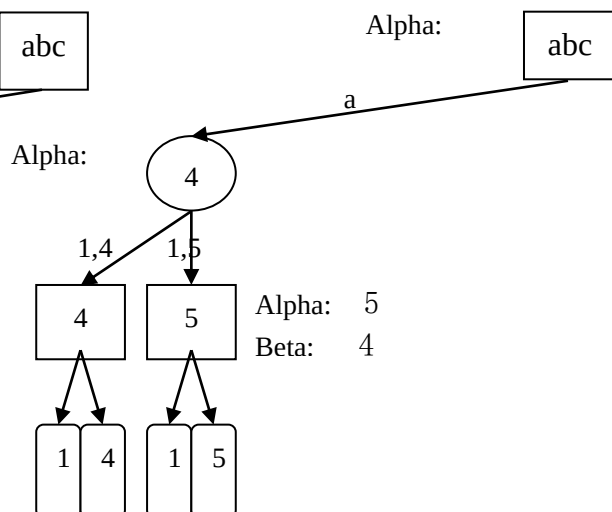


Figure 3 *Alpha – Beta* pruning three

(10) Search the second leftmost node of the third child. After finish see the first leaf node, found that the third child' *Alpha – Beta* , Said this node won't have better solutions, therefore pruning. At the back of the search method is the same as above.

About *Mini max* and *Alpha – Beta* all are symmetrical rotation chess problems for information, abstracting game rules.

(1) Game player. Each time can choose up, down, left and right one of the four strategies. (Some parties may less than four, because some directions will not go). Moving the rear block according to the predetermined logic and merged, pattern conversion to complete.

(2) Internet. In the current arbitrary space placed a square. Placed the new square, pattern transform completed.

(3) Victory conditions: The numerical of every square is 2048.

(4) Failure conditions: Square is filled, and unable to move on to any four direction (all of them cannot touch and combine)

Among 2048, the value of the current pattern is a core of the algorithm, “good” pattern is easy to victory, while “bad” pattern lead to failure, so in order to Increase the probability of victory, and there are several factors to consider:

Monotonic:

Monotonic refers to square’s all directions digital remain increasing or decreasing, this can separate larger and smaller Numbers, So that make the larger or smaller Numbers in order, form a good pattern. As shown in figure 4

8	32	64	512
4	8	16	256
2	4	8	32
		4	8

Figure 4 The 2048 Game of monotonic pattern

Smooth:

Smoothness refers to the Numbers of adjacent grid difference.

The difference should be as small as possible, so that get a better pattern.

Number of Spaces:

When has a full case, game over. Thus the less space has less advantage for player, in order to has a good pattern, Should let spaces as far as possible

Number of isolated space:

Isolation Spaces number refers to the space separate’s degree, if you want to get a good pattern, need to make the square intensive as far as possible. The square more dispersed the pattern poorer.

The above is about 2048 – AI when evaluating pattern, for these inspired index weighting strategy. On the point that pruning the program of choice, in addition to *Alpha – Beta* , taking another pruning on the *mix* point, that only consider the other party move on to the pattern of the worst (actually, the choice of computer is random among 2048), Instead of searching all the other possible way. Because all the choice of the other party is Number of Spaces multiplied two, if search the entire thing will be severely limited actually Search depth. The pruning code see appendix

two.

In terms of search depth, 2048 – AI is never limited the depth of research,

But to limit every time's "thinking". There is set a timeout, the default is 100 ms, during this time, to begin with one, to search what can be achieved. Related code see appendix three.

AI algorithms by running Java can work out the probability of successful synthesis of 2048 more than 90% (91.739%).

5.1.2 The 2048 game analysis and promotion

So, the next game if extended to 2048 $n * n$ the case, can always play it, and the computer (phone) screen can display whether the problem is related to the next "to reach the target number of steps to move the solution of" the problem. Simpler, that is: the last play to 2048 when we moved in the end of the number, or say how many times the sliding screen.

In fact, the resulting 2048 is generated by one of the 2 and 4 together. Here To simplify the discussion, we assume that every move is generated grid 2. In this case, we have the following recurrence relations:

2048 needs to be moved to produce 1 (2 1024 sum)

1024 produced two need to move two times (4 512 respectively adding)

Produced four 512 need to move four times (8 256 were summed)

Produce 8 256 need to move eight times (16 128 respectively adding)

Generating 16 128 need to move 16 times (32 64 respectively adding)

Needed to produce 32 64 32 Mobile (64 32 respectively adding)

Needed to produce 64 32 64 Mobile (128 16 respectively adding)

Need to move to generate 128 16 128 (8 256 respectively adding)

Need to move to produce 256 256 8 (512 4 respectively adding)

Need to move produce 512 4 512 (1 024 2 are summed)

Therefore, the total number of mobile: $512 + 256 + 128 + 64 + 32 + 16 + 8 + 4 + 2 + 1 = 1024 - 1 = 1023$ times.

Several solutions is concomitant circumstances may arise in the course of the game running. For example, the following figure, and the up (or down) at the same time be able to merge four mobile number:

4	2	2	
4	2	2	
4			
4			

Such a merger would clearly reduce the total number of mergers and mobile. But this situation is not always happen, and will happen, but not often move to merge any number of circumstances. Therefore, this parallel movement is not merged and combined in case of the model will not be discussed.

The value 2048 for the mobile number $2048/2 - 1$ times

By law you can get to the theoretical value of the minimum number of moves for 1023 2048 times..

So, for the 2048 games java runs of 91.739% probability of success, in the case of success can be moved by the number of times the average for the 1075 running java, and 1023 the gap is not so effective.

The following statistics give the following 90 players according to the survey, as shown in Table 1

Table 1 Finish the game altogether need mobile number and success

Time s	1026	1109	1083	1047	1038	1048		1025	1102	1033
Yes or no	Y	Y	Y	Y	Y	Y	N	Y	Y	Y
Time s	1049	1033	1147	1232	1098	1132	1034	1065	1024	1101
Yes or no	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y
Time s	1076		1201	1034	1045	1063		1108	1047	1024
Yes or no	Y	N	Y	Y		Y	Y	Y	Y	Y
Time s	1075	1124	1038	1038	1231		1026	1076	1029	1042
Yes or no	Y	Y	Y		Y	N	Y	Y	Y	Y
Time s	1027	1032	1067	1083	1051	1042	1025	1054	1034	
Yes or no	Y	Y	Y	Y	Y	Y	Y		Y	N
Time s	1054	1032	1092	1051	1086	1124	1101		1185	1102
Yes or no	Y	Y		Y	Y	Y	Y	N	Y	Y
Time s		1043	1089	1143	1102	1043	1024	1053	1056	1066
Yes or no	N	Y	Y	Y	Y	Y	Y	Y	Y	Y

Time s	1103	1034	1204	1104	1064		1054	1046	1053	1043
Yes or no	Y	Y	Y	Y	Y	N		Y		Y
Time s	1183	1203		1067	1104	1073	1035	1102	1221	1042
Yes or no	Y		N	Y	Y	Y	Y	Y	Y	Y

Table 2 The single sample statistics

The single sample statistics				
	N	mean	The standard deviation	The mean standard error
VAR0 0001	81	1077.7 654	53.33462	5.92607

Table3 A single sample test

A single sample test						
	Test value = 1075					
	T	df	Sig. (both sides)	The mean difference	The difference of the 95% confidence interval	
					The lower limit	The upper limit
VAR0 0001	.467	80	.642	2.76 543	-9.02 78	14.5 587

Analysis of results:

As shown in table A presenting values the single sample statistics, Including the single sample of the number of cases、 means、 Standard deviation、 Deviation of standard error. The successful completion of 81 times, the average mobile number is 1078 times, the standard deviation is 53.33462, the mean standard error of 5.92607.

The one sample T test results report as shown in table B ,Including t value, degree of freedom (DF) probability, test (Sig), the mean difference, the difference of the 95% confidence interval. The test t value of 0.467, 80 degrees of freedom, 80 degrees of freedom, the probability of Sig. test of the bilateral t (bilateral) value is 2.76543,

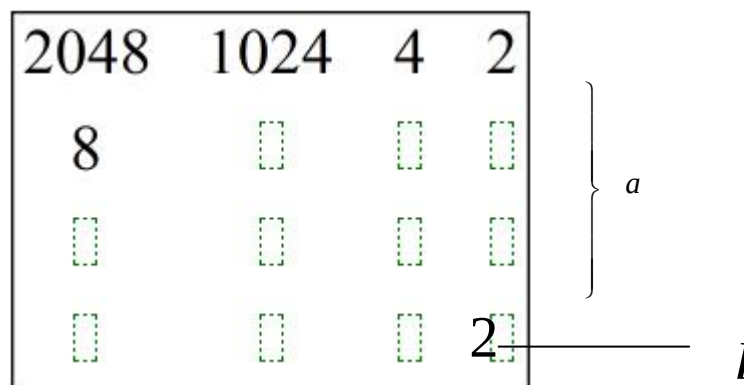
confidence interval [-9.0278,14.5587]. Assuming the game player game success every time the mobile number and the 1075 independent. Because of the significant level of 0.05, and $0.465 > 0.05$, so this assumption does not hold, the number of mobile game player each success with no significant difference between the 1075. The confidence interval formulas corresponding to the T test, the game player each success so moving times of 95% confidence interval = average (1077.7654) 1.96 standard error (5.92607), namely [1066.151089.38], the confidence intervals for the mean and given the difference of [-11.6154,10.6146], this and the upper and lower limits corresponding to the data, but there is a small error.

Through the 2048-AI algorithm in Table 1 data obtained from the Java code running on mobile number and success probability of two results are analyzed, illustrated the effectiveness of the Java operation results, so the data is valid.

5.1.3 Monte Carlo method and 2048 games

In the 2048 game the random variable is the place after 2 or 4 sliding, after each slide in the game player determines what to do next step according to the position and the digital value. Letting the game to continue to reach the 2048 numerical strategy is designated a specific probability digital appear route and inferior position advantage line. The general situation is that advantageous road can combine digital directly, but inferior road has to go another step to do it. In this situation, the probability of success and the numbers of movement under success become general 2048 model.

There are 16 checks of the same size on the desktop, the distance between the place the number slide to and we want to be recorded $a(a > 0)$. The number place after each slide is expressed by 1.tostig it randomly in the parallel lines, and to find the synthesis probability of new number and old number. By Monte carol method we can calculate the probability is $p = \frac{2l}{a\pi}$, through the numbers of movement, we can find the combing numbers of sliding numbers and old numbers. Using combing numbers to eliminate sliding numbers, we get π the approximately number. We can imagine the more we slide the accrete it is.



The qualitative model 2048 times using the Monte Carlo method to analyze the 2048 main game success probability model to quantitative and analytical results of the calculation that cannot be determined. To solve the 2048 games with this method, 2 or

4 is chosen according to specific probability distribution of random variables follow similar values, and sampling method. A variety of distribution of random numbers generated in the game running on a computer will be pseudo random number generated in accordance with the deterministic method

The generation of random Numbers:

Random number is the realization of the random variable values, also named the samples. 2, 4 random results can be used to describe the probability of a discrete random variable function. If we need one or more of the realization of a random variable value (samples), you can use the method of actual movement and then record the result, however, this method has considerable limitations. Using mathematical program to generate the random variable sample, which appear to be random, but it is not really the realization of the random value (samples). The resulting recurrence formula generated, but the proposed random number has no obvious regularity, when given the appropriate scale, they will be close to the uniform distribution of $[0, 1]$ interval, in programming design and package showed RND to represent $[0, 1]$ on random variable of uniform distribution of the sample, and thus it can be constructed to produce other random variables of random Numbers (samples)

For example, from the formula $X = a + (b - a)RND$ given interval $[a, b]$ on the continuous random variable of uniform distribution. If you want to generate the index distribution with parameter random number λ , we can use $X = -(1/\lambda)\ln(RND)$, To generate a mean value of zero and standard deviation of 1 normal distribution, we can use $X_1 = [-2\ln(RND_1)]^{\frac{1}{2}} \cdot \cos(2\pi RND_2)$ and $X_2 = [-2\ln(RND_1)]^{\frac{1}{2}} \cdot \sin(2\pi RND_2)$. giving random X , if $X = \sigma X_2 + \mu$ or $X = \sigma X_1 + \mu$, can get Normal distribution random number of random variables.

Calculating process as follow;

(1) a separate sample parameters $x_1, x_2, \dots, x_n$ are the independent random variables, Expectations and mean square deviation are recorded as $\bar{x}_i$ 和 σ_i . Absolute tolerance is recorded as $\gamma_i = 3\sigma_i$, The relative tolerance as

$$t_i = \frac{\gamma_i}{x_i}, (i = 1, 2, \dots, n); t = (t_1, t_2, \dots, t_n).$$

(2) y the Merge the total number of parameters decided by $y = f(x_1, x_2, \dots, x_n)$, recorded $y = f(x_1, x_2, \dots, x_n)$, Due to the deviation of x_i to $\bar{x}_i$ is very small, we can make Taylor expansion at $\bar{X} \stackrel{\text{def}}{=} (\bar{x}_1, \bar{x}_2, \dots, \bar{x}_N)$ to f . And which

omit three times and more than three times including;

$$y = f(\bar{X}) + \sum_{i=1}^7 d_i (x_i - \bar{x}_i) + \frac{1}{2} \sum_{i=1}^7 e_{ii} (x_i - \bar{x})^2 + \sum_{i=1}^{7-1} \sum_{\substack{i=2 \\ j>1}}^7 e_{ij} (x_i - \bar{x}_i)(x_j - \bar{x}_j) \quad , \text{on which}$$

$$d_i = \left. \frac{\partial f}{\partial x_i} \right|_{\bar{x}}, e_{ii} = \left. \frac{\partial^2 f}{\partial x_i^2} \right|_{\bar{x}}, e_{ij} = \left. \frac{\partial^2 f}{\partial x_i \partial x_j} \right|_{\bar{x}}; \text{ we just use } f \text{ Second expected moment}$$

value and variance, While ignoring the more than three order and third order moment

(3) Because of deviating from the target value y (the position of the number was not the optimal location you want) the number of damage (useless increase mobile number) by the model data can be set and into. The loss to the structural frequency approximation, the analytical expression of the continuous here for pending any constant k .

When there is a large number of mobile digital, thereby it can cause loss of the mobile (4)number of an average value which approximate

expression is $Q(\bar{x}, t) = EL(y) = kE(y - y_0)^2 = k[(EY - y_0)^2 + \sigma_y^2]$ or

$$Q(\bar{x}, t) = k \left\{ [f(\bar{X}) - y_0 + \frac{1}{18} \sum_{i=1}^7 e_{ii} \bar{x}_i^2 t_i^2]^2 + \frac{1}{9} \sum_{i=1}^7 (d_i \bar{x}_i t_i)^2 \right\}$$

(5) Mobile digital decision depends on the tolerance level t_i , regarded $C_i(t_i)$, So

the steps that we needed to complete the game are $C(t) = \sum_{i=1}^n c_i(t_i)$

(6) Complete the 2048 games of the success probability expressed as P_h , The probability of extra mobile times expressed as P_c , $P_c = P\{0.1 < |y - y_0| \leq 0.3\}$.

(7) Comprehensive consideration y deviate from y_0 frequency caused by the loss and appeared, this problem of the objective function is defined as the total number need to be completed the game to finish the game. (Must be on the premise of success)

(8) Being we put the fitting mathematical programming and numerical simulation, using stepwise refinement of iteration method. First of all making sure the calibration values of parameters, the tolerance is determined. Parameters of the calibration value just loss without affecting the number of digit positions. So when we determine the values of parameters calibration, don't consider the numbers appear position, to the number of loss minimum as the goal, USES the formula; Determine the tolerance using (1)~(3) Monte Carlo simulation method, the model is known t value, number of loss expression (5) precision, and then the model was optimized to get what you want.

(9) Using Monte Carlo method can be successful eventually the approximation of

probability is 90.21%, under the premise that the probability of success using build 1078 completed a total of Game 2048 model of mobile games, reaching 2048 value.

Through SPSS data processing can be determined, the average success rate and the mean test times and Java get the success rate of operation and frequency error is small, so the results are effective, mobile number and the probability of success convincing.

5.2 The question of Establishment and solving the problem

Analysis from the probability into consideration, the maximum solution considered in two different conditions. The first kind of circumstance is the last one before the full grid number is 2, the second is the last one before the full grid number is 4.5.2.1 Maximum Solutions Analyze - Finally number two

In the situation of behind Filled squares, the last number happens to be 2 analyzing the Maximum Solutions:

Analyzing it from a simple 4×4 , and then expanding to $N \times N$:



Under the case 2×2 , the maximal solution is $32 = 2^5 = 2^{(2 \times 2 + 1)}$, as Chart shown

16	2
8	4

Under the case 3×3 , The maximal solution is $1024 = 2^{10} = 2^{(3 \times 3 + 1)}$,as Chart shown

512	16	8
256	32	4
128	64	2

Under The case of 4×4 , maximum solution for $2^{16} = 2^{(4 \times 4)}$, which is shown in the chart among them, we know that the minimum solution is 4, so there are $N - 14 = 2$, $N = 16$. In this case if it is fully combined, thus the highest score can be 2^{16} .

2^{16}	2^{12}	2^8	2^1
2^{15}	2^{11}	2^7	2^2
2^{14}	2^{10}	2^6	2^3
2^{13}	2^9	2^5	2^4

After the actual game we can get maximal solution 65536, which is consistent with the calculation results, as shown in figure 6:

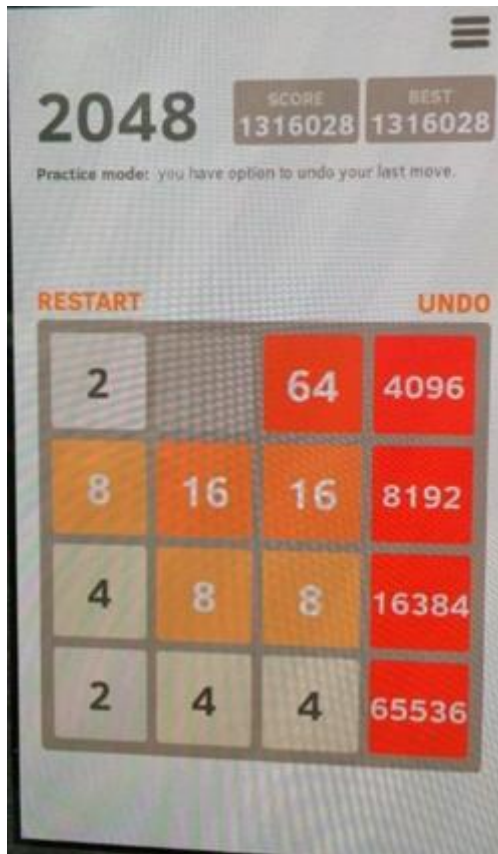


Figure 6 The result of chart 1 is the maximum of 2

And so on, the case of $N * N$, the maximum solution is obviously for the $2^{(N * N)}$.

5.2.2 Maximum Solutions Analyze - Finally number 4

In the situation of behind Filled squares, the last number happens to be 2 analyzing the Maximum Solutions:

Analyzing it from a simple 4×4 , and then expanding to $N \times N$:
Under the Case 1×1 , the maximal solution is 4, as Chart shown

4

Under the case 2×2 , the maximal solution is $32 = 2^5 = 2^{(2 \times 2 + 1)}$, as chart shown,

32	4
16	8

Under the case 3×3 , the maximal is $1024 = 2^{10} = 2^{(3 \times 3 + 1)}$, as chart shown,

2^{16}	2^{12}	2^8	2^1
2^{15}	2^{11}	2^7	2^2
2^{14}	2^{10}	2^6	2^3
2^{13}	2^9	2^5	2^4

Under the case of 4×4 , the maximal solution for $2^{17} = 2^{(4 \times 4 + 1)}$, is shown in the chart .Among them, we know that the minimum solution is 4 so there are $N - 14 = 2$, $N = 16$. In this case if it is fully combined, thus the highest score can be 2^{17} .

2^{17}	2^{10}	2^9	2^5
2^{16}	2^{11}	2^8	2^4
2^{15}	2^{12}	2^7	2^3
2^{14}	2^{13}	2^6	2^2

After the actual game we can get maximum solution is 131072, which is consistent with the calculation results, as it is shown in Figure 7

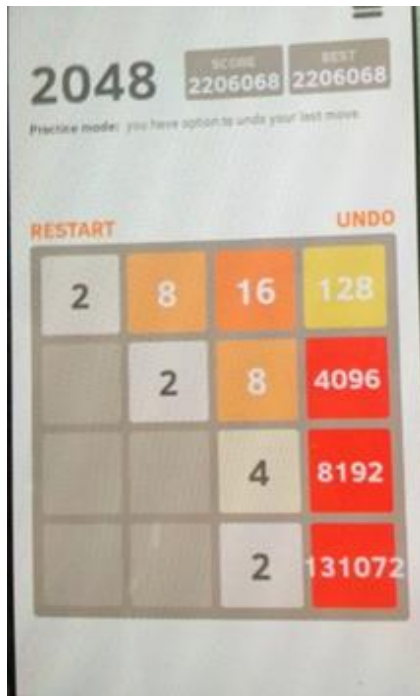


Figure 7 The result of chart 2 is the maximum of 4
And so on, the case of $N \times N$, the maximum solution is obviously $2^{(N \times N + 1)}$.

6. Model's Evaluation and Popularization

6.1 The First Model's Evaluation and Popularization

In order to find out the general model for passing all levels in the Game 2048, the model of double plays can be linked in the process of the first issue. The computers can be treated as the other party, which makes the model more vivid. In the first model, though the algorithm of *Mini max* is a pessimistic algorithm, the player can fully seize the initiative in *Mini max*. If the opponent is perfect in every decision, the player can only reach the expectedly minimum loss. But if the opponent is not perfect in every decision, the player can get better result than before. By Monte Carlo method, probability distributions of influential factor can be calculated. According to the given probability distributions of computer, plenty of random number will be generated. Then we use the random number in the model, the probability distributions of target variables will be calculated. For some issues' idealized analysis, there is a certain gap from reality. Of course, it is because so many accidents are removed that this model can be.

This model solves the issue in random probability, i.e. under uncertainty. Therefore, when people meet some uncertainties in their real life, they can make the most desirable choice.

6.2 The Second Model's Evaluation and Popularization

For the second model, we start from the easiest problem. The relationship

between every increase of cell number and maximum number should be got firstly. Then the regular formula will be found out. After we obtain the relationship between maximum number and cell number, the model is clear. The Game 2048 is added by the generated number two and four. In order to simplify the discussion, we suppose the generated cell is two in every move. The recurrence relations of model will be obtained after idealizing. For this idealization, there is a certain gap in the model. The figure we obtain is only theoretical. In order to solve the issue better, we should collect more reliable data in solving the practical problems.

Meanwhile, this model provides some potential values for game developers and players. For the game developers, they have to set the acceptable difficulty. Then the game will be popular among the users. For the players, they can find the game is interesting by this model. The players are more willing to explore the final result in this game. The meaning of existence for this game is realized.

7. Reference

- [1] 2048 Game, <https://github.com/ov3y/2048-AI>, May 25 2014
- [2] 2048-AI github, <https://github.com/ov3y/2048-AI>, May 25 2014
- [3] An Exhaustive Explanation of Minimax, a Staple AI Algorithm , <http://www.flyingmachinestudios.com/programming/minimax/>, May 25 2014
- [4] Tic Tac Toe: Understanding the Minimax Algorithm , <http://www.neverstopbuilding.com/minimax>, May 25 2014
- [5] CS 161 Recitation Notes - Minimax with Alpha Beta Pruning , <http://cs.ucla.edu/~rosen/161/notes/alphabeta.html>, May 25 2014
- [6] 2048 - AI program algorithm analysis, <http://blog.codinglabs.org/articles/2048-ai-analysis.html>, May 25 2014
- [7] What is the optimal algorithm for the game2048, <http://stackoverflow.com/questions/22342854/what-is-the-optimal-algorithm-for-the-game-2048>, May 25 2014
- [8] 2048 can theoretically unlimited play, <http://m.zhihu.com/question/23492860>, May 25 2014
- [9] Guangting Chen Zheyong Pei, Mathematical Modeling, Beijing, Higher Education Press, Feb.2010

8. Appendix

Code 1 - inspired index by weighted code:

```
// static evaluation function
AI.prototype.eval = function() {
    var emptyCells = this.grid.availableCells().length;
```

```

var smoothWeight = 0.1,
    //monoWeight    = 0.0,
    //islandWeight = 0.0,
    mono2Weight    = 1.0,
    emptyWeight    = 2.7,
    maxWeight      = 1.0;

return this.grid.smoothness() * smoothWeight
    //+ this.grid.monotonicity() * monoWeight
    //- this.grid.islands() * islandWeight
    + this.grid.monotonicity2() * mono2Weight
    + Math.log(emptyCells) * emptyWeight
    + this.grid.maxValue() * maxWeight;
};

```

Code 2— *Alpha – Beta* pruning:

```

// try a 2 and 4 in each cell and measure how annoying it is
// with metrics from eval
var candidates = [];
var cells = this.grid.availableCells();
var scores = { 2: [], 4: [] };
for (var value in scores) {
    for (var i in cells) {
        scores[value].push(null);
        var cell = cells[i];
        var tile = new Tile(cell, parseInt(value, 10));
        this.grid.insertTile(tile);
        scores[value][i] = -this.grid.smoothness() + this.grid.islands();
        this.grid.removeTile(cell);
    }
}

// now just pick out the most annoying moves
var    maxScore    =    Math.max(Math.max.apply(null,    scores[2]),
Math.max.apply(null, scores[4]));
for (var value in scores) { // 2 and 4
    for (var i=0; i<scores[value].length; i++) {
        if (scores[value][i] == maxScore) {
            candidates.push( { position: cells[i], value: parseInt(value, 10) } );
        }
    }
}
}

```

Code 3 - set time to search depth:

```

// performs iterative deepening over the Alpha-Beta search
AI.prototype.iterativeDeep = function() {
    var start = (new Date()).getTime();
    var depth = 0;
    var best;
    do {
        var newBest = this.search(depth, -10000, 10000, 0, 0);
        if (newBest.move == -1) {
            //console.log('BREAKING EARLY');
            break;
        } else {
            best = newBest;
        }
        depth++;
    } while ( (new Date()).getTime() - start < minSearchTime);
    //console.log('depth', --depth);
    //console.log(this.translate(best.move));
    //console.log(best);
    return best
}

```

Code 4-the implementation of 2048 Game's c + +:

```

/*By Reason*/
#include<iostream>
#include <iomanip>
#include<math.h>
#include<stdlib.h>
#include<time.h>
#include <conio.h> // In order to read the direction key
#include<windows.h>
using namespace std;
//srand( (unsigned)time( NULL ) ); // The random number seed can't use here
int pane[4][4]; // checkerboard
int N=1; //2^n
void showpane()// Display board
{
    cout<<setw(46)<<"X2048 by Reason"<<endl;
    cout<<setw(50)<<" |-----|"<<endl;
    for(int i=0;i<=3;i++)
    {
        cout<<setw(24)<<"";
        for(int j=0;j<=3;j++)
        {
            //SetConsoleTextAttribute(GetStdHandle(STD_OUTPUT_HANDLE),FOREGROUND_RED);

```

```

        if(pane[i][j]==0)
            cout<<setw(2)<<"|"<<setw(4)<<" ";
        else
            cout<<setw(2)<<"|"<<setw(4)<<pane[i][j];
        if(j==3)
        {
            cout<<setw(2)<<"|"<<endl;
            cout<<setw(50)<<" |-----|"<<endl;
        }
    }
}

void newgame() //Start the game
{
    N=1;
    for(int i=0;i<=3;i++) // Initialize the board
        for(int j=0;j<=3;j++)
            pane[i][j]=0;
    srand( (unsigned)time( NULL ) );
    int m=rand()%4;
    int n=rand()%4;
    int p=rand()%4;
    int q=rand()%4;
    pane[m][n]=pane[p][q]=2;
    showpane();
}

int if2n(int x)    // To determine whether x is 2n power
{
    int flag=0;
    for(int n=1;n<=11;n++)
    {
        if(x==pow(2,n))
        {
            flag=1;
            if(n>N)
                N=n;
            return flag;
        }
    }
    return flag;
}

int upmove()    // Move up
{
    int flag=0;

```

```

    for(int j=0;j<=3;j++)
        for(int i=0;i<3;i++)
        {
            if(if2n(pane[i][j]+pane[i+1][j])==1)
            {
                pane[i][j]=pane[i][j]+pane[i+1][j];
                pane[i+1][j]=0;
                flag=1;
            }
        }
    return flag;
}

int downmove() // Move down
{
    int flag=0;
    for(int j=0;j<=3;j++)
        for(int i=3;i>0;i--)
        {
            if(if2n(pane[i][j]+pane[i-1][j])==1)
            {
                pane[i][j]=pane[i][j]+pane[i-1][j];
                pane[i-1][j]=0;
                flag=1;
            }
        }
    return flag;
}

int leftmove() // Shift to the left
{
    int flag=0;
    for(int i=0;i<=3;i++)
        for(int j=0;j<3;j++)
        {
            if(if2n(pane[i][j]+pane[i][j+1])==1)
            {
                pane[i][j]=pane[i][j]+pane[i][j+1];
                pane[i][j+1]=0;
                flag=1;
            }
        }
    return flag;
}

int rightmove() // Moves to the right

```

```

{
    int flag=0;
    for(int i=0;i<=3;i++)
        for(int j=3;j>0;j--)
        {
            if(if2n(pane[i][j]+pane[i][j-1])==1)
            {
                pane[i][j]=pane[i][j]+pane[i][j-1];
                pane[i][j-1]=0;
                flag=1;
            }
        }
    return flag;
}

int testup() //whether you move up the test
{
    int flag=0;
    for(int j=0;j<=3;j++)
        for(int i=0;i<3;i++)
        {
            if((if2n(pane[i][j]+pane[i+1][j])==1)&&pane[i+1][j])
            {
                flag=1;
            }
        }
    return flag;
}

int testdown() // whether you move down the test
{
    int flag=0;
    for(int j=0;j<=3;j++)
        for(int i=3;i>0;i--)
        {
            if((if2n(pane[i][j]+pane[i-1][j])==1)&&pane[i-1][j])
            {
                flag=1;
            }
        }
    return flag;
}

int testleft() // whether you move to the left
{
    int flag=0;

```

```

        for(int i=0;i<=3;i++)
            for(int j=0;j<3;j++)
            {
                if((if2n(pane[i][j]+pane[i][j+1])==1)&&pane[i][j+1])
                {
                    flag=1;
                }
            }
        return flag;
    }
    int testright()    // whether you move to the right

    {
        int flag=0;
        for(int i=0;i<=3;i++)
            for(int j=3;j>0;j--)
            {
                if((if2n(pane[i][j]+pane[i][j-1])==1)&&pane[i][j-1])
                {
                    flag=1;
                }
            }
        return flag;
    }
    int panemax()    // The board of maximum number
    {
        int max=pane[0][0];
        for(int i=0;i<=3;i++)
            for(int j=0;j<=3;j++)
                if(pane[i][j]>max)
                    max=pane[i][j];
        return max;
    }
    int ifwin()    // Determine whether victory
    {
        int flag=0;
        if(panemax()==2048)
        {
            cout<<setw(45)<<"You Win!"<<endl;
            flag=1;
        }
        return flag;
    }
    int ifGameOver()    // Determine whether game over

```

```

{
    int flag=0;
    if(testup()+ testdown() + testleft() + testright() == 0)
    {
        cout<<setw(43)<<"Game Over!"<<endl;
        flag=1;
    }
    return flag;
}

void addnewnumberup() // Up after adding a new number
{
    srand( (unsigned)time( NULL ) );
    int n;
    if(N==1)
        n=1;
    else
        n=(rand()%(N)+1);
    int newnumber=pow(2,n);
    for(int i=3;i>=0;i--)
        for(int j=0;j<=3;j++)
            if(pane[i][j]==0)
            {
                pane[i][j]=newnumber;
                return;
            }
}

void addnewnumberdown() // Move down after adding a new number
{
    srand( (unsigned)time( NULL ) );
    int n;
    if(N==1)
        n=1;
    else
        n=(rand()%(N)+1);
    int newnumber=pow(2,n);
    for(int i=0;i<=3;i++)
        for(int j=0;j<=3;j++)
            if(pane[i][j]==0)
            {
                pane[i][j]=newnumber;
                return;
            }
}

void addnewnumberleft()// Add a new number after the shift to the left

```

```

{
    srand( (unsigned)time( NULL ) );
    int n;
    if(N==1)
        n=1;
    else
        n=(rand()%(N)+1);
    int newnumber=pow(2,n);
    for(int j=3;j>=0;j--)
        for(int i=0;i<=3;i++)
            if(pane[i][j]==0)
            {
                pane[i][j]=newnumber;
                return;
            }
}

void addnewnumberright()    // Moves to the right after adding a new number
{
    srand( (unsigned)time( NULL ) );
    int n;
    if(N==1)
        n=1;
    else
        n=(rand()%(N)+1);
    int newnumber=pow(2,n);
    for(int j=0;j<=3;j++)
        for(int i=0;i<=3;i++)
            if(pane[i][j]==0)
            {
                pane[i][j]=newnumber;
                return;
            }
}

int GetDirection()    // Read the directions
{
    int ret = 0;
    do
    {
        int ch = _getch();
        if(isascii(ch))
            continue;
        ch = _getch();
        switch(ch)
        {

```

```

        case 72:
            ret = 2; // top
            break;
        case 75:
            ret = 1; // left
            break;
        case 77:
            ret = 3; // right
            break;
        case 80:
            ret = 4; // down
            break;
        default:
            break;
    }
} while (ret == 0);

return ret;
}
int main() // The main function
{
    system("color f9");
    int makesure=1;
    while(makesure)
    {
        system("cls");
        newgame();
        while(ifwin()+ifGameOver()==0)
        {
            int c=GetDirection();
            switch(c)
            {
                case 2:
                    if(testup()==1)
                    {
                        upmove();
                        addnewnumberup();
                        system("cls");
                        showpane();
                    }
                    break;
                case 4:
                    if(testdown()==1)
                    {
                        downmove();

```

```

        addnewnumberdown();
        system("cls");
        showpane();
    }
    break;
case 1:
    if(testleft()==1)
    {
        leftmove();
        addnewnumberleft();
        system("cls");
        showpane();
    }
    break;
case 3:
    if(testright()==1)
    {
        rightmove();
        addnewnumberright();
        system("cls");
        showpane();
    }
    break;
default:
    break;
}

}
cout<<setw(43)<<" For your final grades: "<<panemax()<<endl;
cout<<setw(60)<<" If you want to restart the game please enter 1, if you
want to end please enter 0."<<endl;
cin>>makesure;
while(makesure!=1&&makesure!=0)
{
    cout<<" Input is not correct, please enter again!"<<endl;
    cin>>makesure;
}
}

cout<<" goodbye ! "<<endl;
system("pause");
return 0;
}

```

Image is obtained by experimental verification:

